

Zero-Copy GEMM-Based Fast Convolution

Caio Salvador Rohwedder
AMD
Calgary, Canada
Caio.Rohwedder@amd.com

João P. L. De Carvalho
AMD
Calgary, Canada
Joao.Carvalho@amd.com

José Nelson Amaral
University of Alberta
Edmonton, Canada
jamaral@ualberta.ca

Abstract—This paper presents ZConv, a new GEMM-based convolution formulation for CPU architectures that eliminates data copies by exploiting the dimension and stride parameters of GEMM library calls. Unlike prior approaches, which reduce convolution to GEMM through Image-to-Column (Im2col) transformations or smaller data copies, this formulation achieves zero-copy execution. An extended version with minimal memory overhead further supports dilated and grouped convolutions, covering most convolution configurations. Extensive evaluation across a large dataset of convolutional layers demonstrates performance that is competitive with state-of-the-art implementations and superior to other published methods. When integrated into PyTorch, the approach yields end-to-end model speedups of up to 1.47 $\times$.

Index Terms—Convolution, Matrix Multiplication, Im2col, BLAS

I. INTRODUCTION

Convolutional layers (abbreviated to convolution in this paper) are the core component in convolutional neural networks (CNNs) and account for the majority of CNN execution time [1]. No single method is optimal for all convolution layers, as hybrid strategies in DL frameworks show. Each approach trades off performance, generality, and portability. This complexity comes from: (i) the many parameters that modify convolution (see Table I); (ii) the tensor layouts of input and output tensors, which are often pre-defined by the framework in use; and (iii) the diversity of execution targets. While Graphics Processing Units (GPUs) enable training and throughput-oriented execution of CNNs, Central Processing Units (CPUs) allow efficient inference for small image batches on a variety of devices. This work targets two-dimensional CNN inference on CPUs. The recent inclusion of matrix multiplication units in CPUs further indicates the relevance of CPU inference; however, these accelerator units also complicate code generation and optimization [2–4].

Direct and General matrix-matrix multiplication (GEMM)-based convolution are the two most widely used implementations, both supported by state-of-the-art DL frameworks such as PyTorch [5] and ONNX Runtime [6]. GEMM-based methods reduce convolution to matrix multiplication, traditionally using the Im2col transformation to leverage the extensive engineering efforts behind optimized GEMM kernels [7]. These kernels are widely available across hardware platforms through BLAS libraries [8–10].

The primary drawbacks of GEMM-based convolution are the additional memory overhead and transformation cost of Im2col. Recent research efforts partially reduce Im2col’s overhead, but

only support a subset of convolution configurations [11–14]. An alternative is to fuse the Im2col transformation directly into the GEMM kernel [15–18], which can effectively eliminate the explicit transformation and achieve strong performance, but it requires modifying the internals of the GEMM implementation. As a result, such methods lack portability across different BLAS libraries and hardware architectures.

This work introduces Zero-Copy Convolution (ZConv), a novel GEMM-based convolution algorithm that fully eliminates the data copies required by Im2col-based approaches, while relying on unmodified GEMM kernels. ZConv is a simple, high-performance, portable, and fully parallelizable algorithm that supports a wide range of convolution configurations. Its core insight is that convolution can be decomposed into multiple GEMM calls that directly access the image tensor by exploiting the stride parameters (*lda*, *ldb*, and *ldc*) accepted by BLAS libraries. By utilizing unmodified GEMM kernels and using conventional input tensor layouts, ZConv can leverage multiple BLAS libraries and is easily integrated into existing deep learning frameworks. ZConv’s two outermost loops have no dependencies between iterations, and thus can be trivially parallelized. Additionally, a low-memory-overhead extension of ZConv is proposed to support dilated and grouped convolutions. Together, these algorithms cover the most convolution configurations.

Performance evaluation with the large dataset introduced by Alvarenga et al. [19], which includes over 9000 unique convolutional layers from more than 1200 vision models, reveals that ZConv outperforms both the prior work and the state-of-the-art convolution implementations used by PyTorch. This large-scale evaluation enabled the design of simple heuristics based on convolution parameters to decide when to utilize ZConv over alternative methods to maximize model performance. Integrating ZConv into PyTorch’s hybrid convolution implementation for end-to-end model evaluation results in inference speedups of up to 1.8 $\times$ over unmodified PyTorch.

In summary, this work makes the following contributions:

- (i) ZConv, a novel GEMM-based convolution algorithm that leverages the *lda*, *ldb*, and *ldc* parameters of the BLAS API to eliminate the Im2col transformation and use zero copies (Section III-A).
- (ii) An extensive evaluation of ZConv against prior and state-of-the-art methods in standalone layers (Section IV-B)
- (iii) An evaluation of end-to-end vision models where ZConv was integrated into PyTorch’s hybrid convolution imple-

TABLE I: Notation for convolution parameters.

N:	Batch size	IC:	Input channels
IH:	Input height	IW:	Input width
OC:	Output channels	OH:	Output height
OW:	Output width	FH:	Filter height
FW:	Filter width	SH:	Stride height
SW:	Stride width	DH:	Dilation height
DW:	Dilation width	PH:	Padding height
PW:	Padding width	GR:	Groups

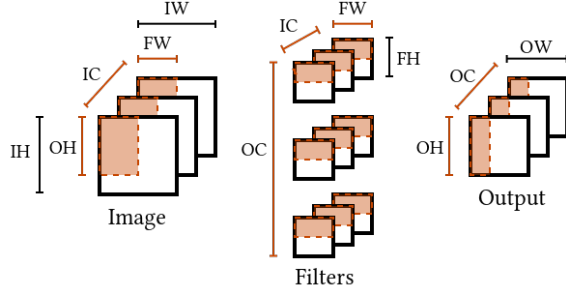


Fig. 1: Convolution tensors and their respective dimensions (omitting batch). Highlighted in orange, an example of the inputs for a GEMM call within ZConv.

mentation (Section IV-C).

- (iv) A publicly available artifact containing all implementations and evaluation scripts.¹

II. CONVOLUTION AND GEMM

Convolution takes three input tensors: the input image, filters, and output (illustrated in Figure 1), along with an optional bias array. These four-dimensional tensors can have different data layouts, that is, the order in which their dimensions are stored in memory. In addition to the input tensors, convolution utilizes several parameters defined in Table I that modify its behavior.

GEMM is defined in (1), where α and β are scalar values, and $C_{m \times n}$, $A_{m \times k}$, and $B_{k \times n}$ are the matrix operands. This API uses the following parameters: (i) *layout*: the memory layout of matrices (row-major or column-major), (ii) *transx*: whether to transpose matrix x before multiplication, (iii) m , n , k : the dimension sizes of the matrices, (iv) *alpha* and *beta*: the scalar multiplier parameters, (v) *ldx*: the leading dimension for matrix x . For row-major matrices, *ldx* is the number of elements in memory between consecutive rows, (vi) a , b , c : the base pointers to the matrices.

$$C_{m \times n} = \beta C_{m \times n} + \alpha A_{m \times k} \cdot B_{k \times n} \quad (1)$$

While the Im2col approach transforms the convolution input image into a new contiguous matrix, the GEMM API supports non-contiguous input matrices. In the row-major layout, the GEMM API only assumes that elements within a row are contiguous. The stride between rows, the row size, and the

starting position of the matrix are all flexible and configurable through the parameters previously mentioned. The column-major layout operates similarly, but with contiguous columns instead of rows. BLAS libraries rely on tiling—dividing input matrices into smaller submatrices (tiles)—and packing matrices into contiguous buffers to optimize cache locality [10, 20]. These transformations also allow GEMM to achieve high performance with non-contiguous inputs. ZConv leverages GEMM’s API flexibility by providing non-contiguous matrices directly to GEMM.

III. ZERO-COPY CONVOLUTION

Zero-Copy Convolution (ZConv) is a novel convolution algorithm for CPUs designed around the key insight that all data copies required by traditional Im2col-based approaches can be eliminated. This is achieved by decomposing the convolution into multiple GEMM calls and leveraging the GEMM API to operate on non-contiguous input matrices. ZConv’s performance relies on heavily optimized GEMM kernels, making its algorithm simple and compatible with most architectures via open-source or vendor-specific BLAS libraries. ZConv supports all convolution parameters used in PyTorch’s *Conv2D* operator (listed in Table I), except for the padding mode, which is fixed to zero-padding. To provide this support, ZConv includes two versions: a zero-copy implementation that covers most common configurations (Section III-A), and an extended version that introduces a small copy step to support grouped and dilated convolutions (Appendix A). The start of Algorithm 1 specifies the required tensor layouts for ZConv, which are commonly found in frameworks such as PyTorch [5] and TensorFlow [21]. To execute ZConv in end-to-end models, its algorithm was integrated into PyTorch, as described in Section III-B.

A. ZConv Algorithm

In Algorithm 1, ZConv is described; its parameters are given in Table I. The two outermost loops iterate over the batch (N) and output width (OW), which correspond to the first and third dimensions of the output tensor. Each iteration of these loops computes a distinct output tile of shape $[OH, 1, OC]$, allowing them to be parallelized. Lines 3-7 initialize the output tile to the corresponding bias values (or to zero if there is no bias). The only remaining loop, at Line 8, iterates over the filter height (FH), setting the appropriate GEMM parameters and calling GEMM. Each iteration over the filter height computes a partial result for the $[OH, 1, OC]$ output tile. GEMM is called in row-major layout, with input matrices used as-is (*i.e.*, no transposition), and with scalar multipliers *alpha* and *beta* both set to one. Line 14 describes the GEMM dimensions m , n , and k typically used in ZConv. An example of these matrix tiles within the convolution tensors is highlighted in Figure 1. Overall, GEMM is invoked $N \times OW \times FH$ times.

Padding is omitted from Algorithm 1 for readability. Convolution with padding is handled in ZConv by skipping accesses to padding elements, resulting in GEMM calls with smaller matrices at the image edges. Padding elements can be skipped

¹Will be packaged upon publication, but the code was submitted as supplementary material.

Algorithm 1 ZConv (padding omitted for brevity).

Require: $image[N, IH, IW, IC]$, $filter[FH, FW, IC, OC]$
 $output[N, OH, OW, OC]$, $bias[OC]$

```

1: for  $n \leftarrow 0$  to  $N - 1$  do ▷ Parallel loop
2:   for  $ow \leftarrow 0$  to  $OW - 1$  do ▷ Parallel loop
3:     for  $oh \leftarrow 0$  to  $OH - 1$  do
4:       for  $oc \leftarrow 0$  to  $OC - 1$  do
5:          $output[n, oh, ow, oc] \leftarrow bias[oc]$ 
6:       end for
7:     end for
8:     for  $fh \leftarrow 0$  to  $FH - 1$  do
9:        $A \leftarrow \text{addr}(image[n, fh, ow \cdot SW, 0])$ 
10:       $B \leftarrow \text{addr}(filter[fh, 0, 0, 0])$ 
11:       $C \leftarrow \text{addr}(output[n, 0, ow, 0])$ 
12:       $layout \leftarrow \text{rowMajor}$ 
13:       $transa \leftarrow \text{noTrans}, transb \leftarrow \text{noTrans}$ 
14:       $m \leftarrow OH, k \leftarrow FW \cdot IC, n \leftarrow OC$ 
15:       $lda \leftarrow IW \cdot IC \cdot SH, ldb \leftarrow n, ldc \leftarrow OW \cdot OC$ 
16:       $\alpha \leftarrow 1, \beta \leftarrow 1$ 
17:       $C \leftarrow \alpha AB + \beta C$  ▷ GEMM call
18:    end for
19:  end for
20: end for

```

and never instantiated in temporary buffers because the padding mode is fixed to zero-padding in ZConv. This is the default mode in PyTorch. A full implementation with padding handling will be provided in the artifact.

Figure 2 illustrates the example of the ZConv algorithm with the convolution parameters detailed in its caption. The figure expands the iterations over output width and filter height, corresponding to Lines 2 and 8 of Algorithm 1. The highlighted tiles in the image and filter represent the elements accessed during each iteration, which serve as inputs to the GEMM operation shown on the right side of the figure. This GEMM accumulates its result into the corresponding tile of the output. No data copies are involved in selecting the GEMM input tiles—these tiles are determined using a combination of their base pointers, matrix dimensions, and leading dimensions.

Intuitively, each GEMM call in ZConv is equivalent to sliding a single filter row vertically across the image. In Figure 2, the first filter row is never multiplied with the elements of the last image row, which is why the image tiles do not cover the full column height in this example. Similarly, the second filter row is not multiplied with the elements of the first image row. The horizontal iterations in the image follow the width stride.

Algorithm 1 sets the GEMM parameters in Lines 9-16. Matrix A points to the beginning of an $[OH, FW \times IC]$ tile of the image tensor. Matrix A is a non-contiguous tile, where each row contains $FW \times IC$ contiguous elements, and the lda parameter specifies the stride between rows. For example, in Figure 2, the highlighted tiles from the image tensor are obtained with: $m \leftarrow 3$, $k \leftarrow 2$, and $lda \leftarrow 4$. The value of lda represents the stride in memory between the rows of the image tiles. The algorithm naturally produces a matrix C of shape $[OH, OC]$. While this matrix could be stored as a contiguous tile in the output tensor, it would cause the output to have a transposed layout where the OH and OW dimensions

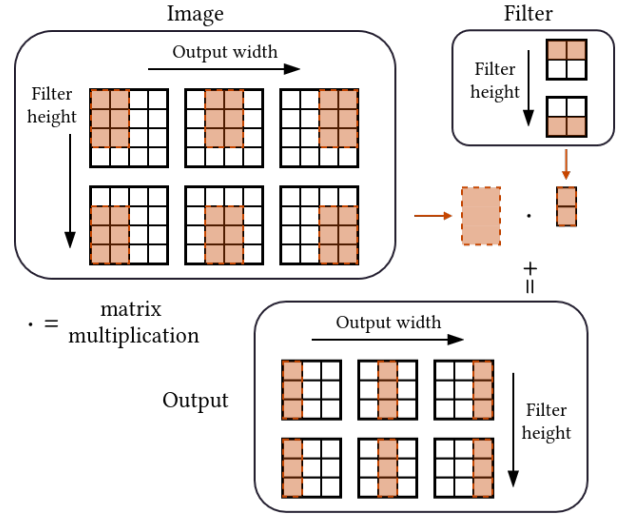


Fig. 2: View of ZConv in a simplified convolution that uses unit stride, no padding, no dilation, and a 2×2 filter. The tensors have only height and width dimensions, the image is 4×4 , and the output is 3×3 . Black arrows indicate loop iterations, and colors indicate the data flow.

are swapped. Matrix C is stored as a non-contiguous tile (using the ldc parameter) to avoid this transposed layout. In contrast, matrix B is a contiguous tile from the filter tensor. Although the filter tile in Figure 2 looks transposed, matrix B does not require transposition because a single row and a single column both appear as contiguous arrays in memory. This remains true for the algorithm when considering the additional filter dimensions.

B. Integration of ZConv with PyTorch

PyTorch [5] supports multiple convolution backends and includes a selector that chooses between them, generally based on convolution parameters [22]. ZConv was integrated as a new backend that is selected when the following conditions are met in end-to-end vision models: (i) BLAS libraries are enabled (to provide a GEMM implementation). (ii) The input image uses the channel-last layout $[N, IH, IW, IC]$ [23], matching the layout expected by ZConv. (iii) The convolution is not grouped ($GR = 1$), not pointwise ($FH \neq 1 \vee FW \neq 1$), and both output height and width dimensions are smaller than the number of input channels ($OH < IC \wedge OW < IC$). The motivation for this heuristic is described in Section IV-B3. A search-based approach could further refine this heuristic, but it is out of the scope of this paper.

The filter tensor in the PyTorch channel-last layout $[OC, FH, FW, IC]$ differs from the layout used by ZConv, $[FH, FW, IC, OC]$. To use the ZConv backend, the filter tensor must be transformed. This transformation can be performed either once ahead-of-time when converting the model to the channel-last format or before each invocation of a ZConv convolution. The end-to-end model evaluation performs the layout transformation ahead of time (Section IV-C).

IV. EXPERIMENTAL EVALUATION

This evaluation demonstrates that ZConv outperforms previous approaches and achieves competitive results compared to state-of-the-art implementations, with performance gains depending on the convolution parameters. In end-to-end models, ZConv improves inference execution time by up to $1.47\times$. Section IV-B discusses standalone layer performance and Section IV-C discusses end-to-end model inference.

A. Setup

Table II provides details of the experimental machine configuration. Experiments run on performance (P) cores, and multithread runs use all 8 P-cores; the governor is set to *performance*, frequency locked at 3 GHz, and Hyper-Threading and Turbo Boost are disabled. The system runs Ubuntu 22.04.5 with kernel 6.5.0. Table III lists the software versions used, with additional details available in the accompanying artifact.

Two collections of vision models are used: *TorchVision* [24] and *Timm* [25], both of which provide models for PyTorch, including architectures such as MobileNet [26], ResNet [27], and EfficientNet [28]. Table IV summarizes the number of unique models in each collection and the distinct convolution layers they contain. TorchVision models that do not take images as input (video and optical flow models) are not included in Table IV, and quantized models are omitted because they do not use PyTorch’s standard *Conv2D* implementation.

B. Standalone Convolutions

This experiment evaluates ZConv by measuring the execution time of the standalone convolutional layers from Table IV. The evaluated convolution methods are listed below and Table V summarizes the input image, weight, and output layouts they use. Conversion to these layouts is not included in the execution time measurements. For methods that rely on GEMM calls, Intel’s OneMKL is used as the default BLAS library. Before measuring performance, the correctness of these methods was verified using LibTorch as the reference output.

- 1) **Yaconv**, a convolution method introduced by Korostelev et al. [14] as an extension of the BLIS library. It uses the author’s original implementation, with modifications to prevent runtime errors and to remove extra generated elements that are not part of the output.
- 2) **Im2col**, a C implementation based on the Caffe framework that transforms convolution into GEMM operations through the Im2col data transformation [7, 29]. Parallelism is handled by the GEMM library.
- 3) **ZConv**, a C implementation of Zero-Copy Convolution that leverages OpenMP for parallelism. It also utilizes OneMKL’s JIT optimization for small GEMM sizes, which is handled automatically by the library.
- 4) **LibTorch**, PyTorch’s C++ API of *Conv2D*, which is used when running models with convolutional layers. It may call multiple convolution backends, including OneDNN. OneDNN supports multiple convolution algorithms, but

in LibTorch, it is set to use direct convolution implementations. LibTorch also allocates its output tensor as part of the computation.

- 5) **LibTorch-ZConv**, the ZConv algorithm integrated into PyTorch, similar to **LibTorch**, also allocates the output tensor.

The experiment runs convolutions with a custom benchmark based on Google Benchmark [30] and ConvBench [19]. Methods run layers in random order; each configuration runs at least ten times (or until 0.5 seconds) with a 0.1 second warm-up, and the mean over five or more repeats is reported. Additional repetitions are done when 95% confidence intervals (CI) overlaps between two compared points and differences exceeded 1%. If the CI overlaps and the difference is less than 1%, the comparison is considered insignificant.

In the following comparison graphs (Figure 3), the bars represent the speedup of one method over another across different convolution layers. The legend indicates the number of speedup and slowdown cases, while the x-axis label specifies the total number of layers shown. The y-axis uses a logarithmic scale to accommodate the wide range of values. Unlike traditional speedup graphs, which restrict slowdown values between 1 and 0, both speedup and slowdown are unbounded and symmetrical. The bars are sorted by speedup. Speedup values are clipped at the 0.99 quantile, with the maximum value displayed at the top of the graph. Similarly, slowdown values are clipped at the 0.01 quantile, with the minimum value shown at the bottom of the graph.

The total number of layers shown on the x-axis may differ from the total in Table IV due to omitted layers. Two conditions lead to the omission of a layer: Layers where the execution time difference is insignificant, and layers filtered by a heuristic because executing them with the new method is not beneficial. The caption of Figure 3 specifies heuristic-based exclusions.

A heuristic applied in all comparisons is the exclusion of pointwise layers (6072 layers from the total in Table IV). This is because pointwise convolutions can be computed directly as GEMM without additional transformations (e.g., Im2col). ZConv is designed to extract multiple smaller GEMMs within convolutions that require the Im2col data layout transformation. Direct GEMM, when applicable, is a simpler and more efficient approach.

Furthermore, six representative convolution layers, described in Table VI, are used to illustrate the memory hierarchy effects of the compared methods. Layers 1 to 3 are more conventional convolution configurations with decreasing input height and width. Layer 4 is a strided convolution that does not overlap image patches. Layer 5 is a grouped (depthwise) convolution, and Layer 6 represents a dilated convolution. For the tables with memory metrics in this section, results were collected using the Linux Perf tool [31]. The methodology for Perf runs was adjusted so that all compared methods execute the same number of iterations, with a minimum of one hundred iterations. Reported Perf events are the average of one hundred such runs, which were run on a single thread for better measurement accuracy.

TABLE II: Experimental configuration

CPU	i7-14700K
Cores Used	8 P-Cores
Frequency	3 GHz (locked)
Memory	64 GiB DDR5

TABLE III: Software versions

PyTorch	2.5.1
OneDNN	3.5.3
OneMKL	2025.0.0
BLIS	1.0

TABLE IV: Models and convolutional layers

	TorchVision	Timm
Models	98	1149
Unique <i>Conv2D</i>	9826	

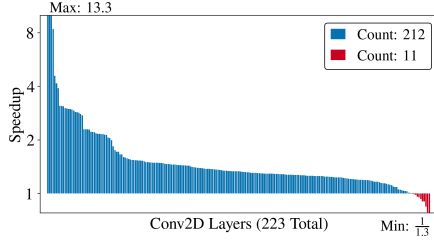
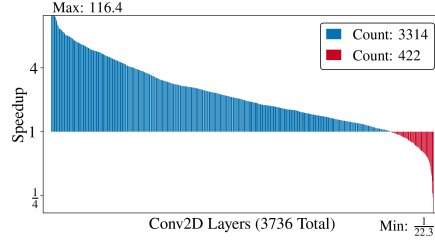
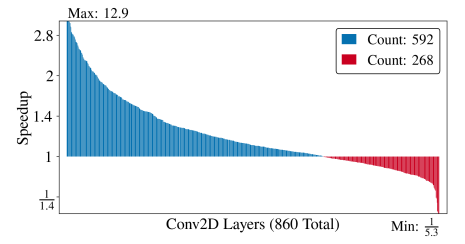
(a) ZConv (BLIS as BLAS) over **Yaconv**(b) ZConv over **Im2col**(c) LibTorch-ZConv over **LibTorch**

Fig. 3: Improvement of ZConv over multiple convolution algorithms on individual *Conv2D* layers. All graphs exclude pointwise layers. Yaconv graphs exclude layers that are unsupported by Yaconv, and for which Yaconv generated incorrect results. The LibTorch graph excludes grouped layers and follows a heuristic defined in Section IV-B3. Executions are multithreaded, except for the Yaconv comparison.

TABLE V: Tensor layouts used by evaluated methods.

	Image/Output Layout	Weight Layout
Im2col	N, C, H, W	OC, IC, FH, FW
Yaconv		
ZConv	N, H, W, C	FH, FW, IC, OC
LibTorch-ZConv		
LibTorch	N, H, W, C	OC, FH, FW, IC

TABLE VI: Parameters for convolution layers used in memory hierarchy metrics comparison. Batch is set to 1.

	IC	IH	IW	OC	FH,FW	PH,PW	SH,SW	DH,DW	GR
#1	32	112	112	64	3,3	1,1	1,1	1,1	1
#2	64	56	56	64	3,3	1,1	1,1	1,1	1
#3	224	7	7	224	3,3	1,1	1,1	1,1	1
#4	320	14	14	320	2,2	0,0	2,2	1,1	1
#5	384	14	14	384	3,3	1,1	1,1	1,1	384
#6	960	33	33	256	3,3	12,12	1,1	12,12	1

1) *Yaconv*: Figure 3a compares ZConv against Yaconv. For this comparison, ZConv is executed with a single thread because Yaconv does not support multithreading. Yaconv does not handle strided, dilated, or grouped convolutions; additionally, out of the 6369 *Conv2D* layers supported by Yaconv, 425 had incorrect results. These layers were excluded from the comparison. As Yaconv is integrated into the BLIS library, ZConv also utilizes BLIS as its BLAS provider for an accurate comparison of both methods. The results show significant improvements across almost all *Conv2D* layers, with a maximum speedup of $13\times$. Table VII compares memory hierarchy metrics for the two layers from Table VI, which are supported by Yaconv. Bolded values highlight metrics improved by ZConv.

Three main factors contribute to these improvements. (1) Due to its packing decisions, Yaconv performs additional computations that introduce extraneous elements in its output, requiring a temporary buffer to filter them out. In contrast, ZConv does

TABLE VII: Metrics of **Yaconv** / **ZConv** using BLIS as BLAS

Layer	Execution		Load Misses				Page Faults
	Time	Instr.	L1	L2	L3	dTLB	
1	1.3	1.3	0.8	8.4	16.6	77.5	278.0
2	1.3	1.3	0.2	17.2	26.4	70.4	345.6

TABLE VIII: Metrics of **Im2col** / **ZConv** (Single-thread)

Layer	Execution		Load Misses				Page Faults
	Time	Instr.	L1	L2	L3	dTLB	
1	1.1	1.1	1.2	20.2	448.9	48456.0	13.7
2	1.1	1.3	0.4	15.1	93.3	16106.3	11.9
3	1.5	1.7	0.9	0.3	3.9	4.6	1.6
4	1.0	1.4	0.2	0.1	0.5	2.8	1.4
5	0.2	0.3	0.4	312.9	137.3	6.2	7.0
6	2.0	1.8	0.3	0.2	94.7	0.8	181.8

not perform unnecessary computations. This factor is evidenced by ZConv executing fewer instructions, even though both algorithms have a similar design. (2) All optimizations (e.g., loop ordering) in Yaconv, besides the GEMM microkernel, are manually implemented. ZConv delegates most of these optimizations to the GEMM kernel, benefiting from its more advanced and flexible optimization strategies. (3) Lastly, Yaconv is tightly integrated with the BLIS library, whereas ZConv is BLAS-agnostic. This flexibility allows ZConv to leverage vendor-optimized libraries such as OneMKL, which can enable even better speedups than the ones shown in Figure 3a.

2) *Im2col*: Figure 3b compares ZConv against Im2col with multithreaded execution. Table VIII shows the comparison of memory metrics between Im2col and ZConv for single-threaded execution. ZConv generally outperforms Im2col due to its effective parallelization strategy. Im2col relies on GEMM calls for parallelism, but its loops around GEMM (over the batch dimension and the number of groups), and the Im2col transformation run sequentially. In contrast, ZConv parallelizes

TABLE IX: Metrics of **LibTorch** / **LibTorch-ZConv** (Single-thread)

Layer	Execution		Load Misses				Page Faults
	Time	Instr.	L1	L2	L3	dTLB	
1	0.9	0.9	0.3	1.9	0.4	0.7	0.9
2	1.0	0.9	0.4	1.5	0.4	0.7	1.2
3	1.3	1.1	2.7	2.9	0.3	0.2	1.0
4	1.4	1.0	2.5	2.5	2.0	6.0	1.0
5	0.1	0.1	0.3	1.1	1.9	0.5	0.9
6	2.4	2.0	8.4	3.1	17.2	1.7	36.4

its two outermost loops, ensuring that its entire computation runs in parallel. More advanced Im2col-based implementations, such as those used in LibTorch, improve parallelism by distributing the Im2col transformation across multiple threads.

The slowdown cases in Im2col comparisons can be mostly attributed to strided layers. In these layers, the Im2col transformation copies fewer or no redundant elements, reducing its overhead. Layer 4 in Table VIII is one such example, where Im2col has a reduced number of misses in all cache levels compared to ZConv. Layer 5 is another disadvantage case for ZConv, because it is a grouped (depthwise) layer that runs in a single thread. Increasing the number of groups reduces the size of the matrices in GEMM and increases the GEMM calls, therefore reducing the GEMM effectiveness. ZConv achieves better performance in the remaining cases by executing with no copies, redundant or otherwise, and still leveraging the highly optimized GEMM kernel, as evidenced by the improved memory hierarchy metrics for the remaining layers.

3) *LibTorch*: Figure 3c compares LibTorch-ZConv against LibTorch with multithreaded execution. This figure excludes grouped convolutions for the reasons mentioned in Section IV-B2. It also excludes layers where the output height or output width is greater than or equal to the number of input channels ($OH \geq IC \vee OW \geq IC$). Out of the 1384 total layers (773 speedups, 611 slowdowns), 860 layers (592 speedups, 268 slowdowns) remain after applying this additional heuristic. The results show overall good performance improvements even with this stronger baseline. Table IX shows the comparison of memory metrics between Libtorch and LibTorch-ZConv for single-threaded execution.

To understand the added heuristic (excluding layers where $OH \geq IC \vee OW \geq IC$), recall that GEMM is called $FH \times OW \times N$ times in ZConv, and the GEMM dimensions (assuming no padding) are: $M = OH$, $K = FW \times IC$, $N = OC$. The filtered layers have an increased number of irregular GEMMs, leading to lower performance. GEMM operations in linear algebra libraries are known to be less efficient on inputs with irregular sizes, such as having a small K dimension [32], and data reuse is lost between GEMM calls, so having more calls increases the computation overhead. A large majority of the filtered layers are the first few convolutional layers of the vision models, where the output height is large (e.g., 112), but the number of input channels is small (e.g., 3 in an RGB image). One such example is Layer 1 in Table IX, where LibTorch has better performance and better metrics in all memory levels but L2

cache compared to LibTorch-ZConv.

LibTorch primarily utilizes its OneDNN backend, which is used for all layers in Table IX, except Layer 3. OneDNN achieves great performance results from its highly optimized and target-specific templates designed for convolution [33–35]. ZConv outperforms LibTorch’s OneDNN backend in Layers 4 and 6 of Table IX. In Layer 6, LibTorch-ZConv shows significant improvements over all memory metrics, largely due to its efficient handling of the big padding parameters in this layer, where padding elements (zeros) are never instantiated in temporary buffers. LibTorch also employs its native Im2col-based backend for some convolution configurations. Layer 3 in Table IX is one example. As a result, the memory metrics for Layer 3 follow the same trends observed in the Im2col comparison. Note that the LibTorch method could be modified to receive as input an image in the MKLDNN layout (using the `to_mkldnn` helper). However, that would restrict the backend choice of LibTorch to OneDNN, and the results remain very similar to Figure 3c.

C. End-to-end Models

This experiment evaluates the impact of integrating ZConv in PyTorch as an additional convolution backend on end-to-end model inference time. Unlike the standalone convolution evaluation in Section IV-B, which uses a C-based benchmarking framework, this section evaluates model inference using PyTorch’s Python API. The Python API is preferred as it enables running models from TorchVision and Timm in Table IV. The models were evaluated under the following configurations:

- 1) **PyTorch**: PyTorch’s default implementation. The model and input image are converted to the channel-last memory format before measurements.
- 2) **PyTorch-ZConv**: The previous configuration with the new convolution backend implementing ZConv to PyTorch, integrated as described in Section III-B.

Each model is executed five times for warm-up, followed by fifty timed runs for both methods. The execution time of each non-warm-up run is recorded. This entire process is repeated five times or more to ensure consistent results. Additional repetitions follow the same methodology as in Section IV-B regarding confidence intervals and insignificant differences. All models listed in Table IV are run in inference mode using their respective default input sizes. All tests are executed with a batch size of 1.

The graphs in Figure 4 follow a similar format to those in Section IV-B. However, in this case, the y-axis is in a linear scale, with the x-axis label indicating the total number of models shown. All models are included in the graphs except when the execution time difference between the two methods is insignificant, following the same methodology as in the previous section.

Figures 4b and 4a compare PyTorch-ZConv against PyTorch across Timm and TorchVision models for multithreaded execution. Notably, the speedup ranges vary between graphs, with wider ranges present in the TorchVision results. Overall, the speedup trends are less pronounced than in Section IV-B

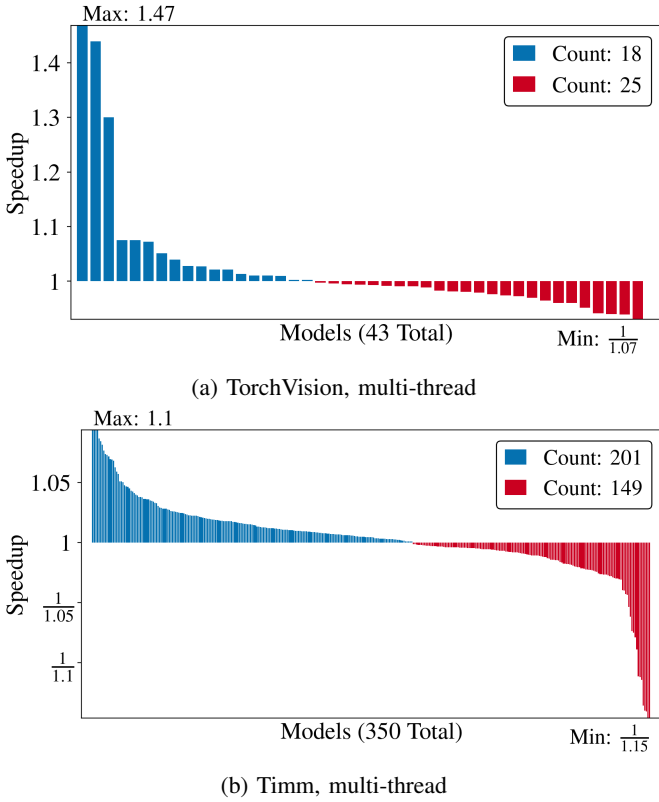


Fig. 4: Improvement of **PyTorch-ZConv** over PyTorch on end-to-end models.

due to additional operations present in these models, and the fact that not all convolutional layers utilize ZConv. Across all results, the highest observed speedup is $1.47\times$, while the largest slowdown is $1.15\times$. The impact of ZConv on performance depends on the specific combination of convolutional layers present in each model.

V. RELATED WORK

Common approaches to optimizing convolution can be categorized as direct convolution and methods based on general matrix-matrix multiplication (GEMM), the Fast Fourier Transform (FFT), and Winograd’s minimal filtering algorithm. FFT-based convolution is sensitive to small filter sizes, whereas Winograd-based methods often introduce numerical instability [36–38]. Direct convolution approaches use analytical performance models akin to those used in high-performance GEMM implementations [39–41], however, generalizing them across different microarchitectures is both time-consuming and challenging. This work introduces ZConv, a novel GEMM-based convolution algorithm.

MEC [11], SMM-Conv [12], and Yaconv [14] reduce convolution to GEMM similarly to ZConv. MEC reduces vertical redundancies present in Im2col. SMM-Conv and Yaconv observe that implicit smaller matrix multiplications arise when isolating a filter row (or column), which can be executed as GEMM operations. ZConv addresses several limitations

found in these prior works: (i) MEC and SMM-Conv require additional temporary buffers for their execution. Yaconv’s output includes extraneous elements. In the same convolution configurations supported by these approaches, ZConv requires no additional buffers. (ii) Yaconv, SMM-Conv, and MEC don’t fully support dilated, grouped, and strided convolutions. ZConv supports all of them. (iii) Yaconv’s algorithm is complex and tightly coupled to the BLIS library. SMM-Conv, on the other hand, does not utilize a GEMM kernel, likely due to the small size of the matrices it multiplies. ZConv is a more portable approach, relying entirely on GEMM kernels for optimizations and is compatible with any BLAS library. (iv) Yaconv does not support parallel execution, whereas SMM-Conv and MEC may utilize separate parallel steps in their execution. In contrast, ZConv is fully fused into a single loop nest and trivially parallelized. (v) Lastly, only ZConv is evaluated in a large set of convolution layers, which enables the design of a heuristic to integrate it into a hybrid convolution approach, such as that used by PyTorch (Section IV-B3).

Anderson et al. [13] showed that conventional convolution can be decomposed into $F_H \times F_W$ pointwise convolutions that do not rely on Im2col. However, the two algorithms proposed in their work require additional memory, depend on the parallelism offered by GEMM kernels, and require the image tensor to be shifted or to have certain elements temporarily zeroed between GEMM calls. ZConv requires no auxiliary memory and no extra operations beyond GEMM. Moreover, while Anderson et al. [13] do not address how their methods can support strided, padded, dilated, or grouped convolutions, ZConv is designed to handle all of these convolution variants.

Another approach used in prior work is to fuse the Im2col transformation into the GEMM operation itself [15–18]. Dukhan [16] introduced the Indirect Convolution, which modifies the GEMM routine used in an Im2col implementation to access the image tensor directly via an indirection buffer. San Juan et al. [17] integrate Im2col into the packing step of GEMM, generating packed tiles directly from the image tensor. While these methods reduce the overhead associated with Im2col, they require modifications to the GEMM implementation. In contrast, ZConv eliminates the Im2col transformation and relies on unmodified GEMM kernels that access the image tensor directly.

VI. CONCLUSION

GEMM-based convolution remains a popular approach due to the widespread availability and heavy optimization of BLAS libraries, despite the computational and memory overhead of the commonly used Im2col transformation. This work introduces the ZConv algorithm, which removes the need for Im2col by enabling GEMM calls to access convolution inputs directly. ZConv improves upon prior GEMM-based methods by requiring no data copies, relying on unmodified GEMM kernels, and supporting a wide range of convolution configurations. As demonstrated in the evaluation, ZConv achieves significant speedups over prior work and state-of-the-art PyTorch implementations. When integrated into PyTorch,

ZConv achieves up to a $1.47\times$ speedup in end-to-end model inference, being particularly effective in image segmentation models that utilize dilated convolution. This work highlights the potential to leverage existing high-performance GEMM kernels in an unconventional manner: by exploiting non-contiguous matrix inputs via its `ldx` parameters. Future work includes adapting the algorithm to target GPU execution.

REFERENCES

- [1] Y. LeCun, B. Boser, J. S. Denker, D. Henderson, R. E. Howard, W. Hubbard, and L. D. Jackel, "Backpropagation applied to handwritten zip code recognition," *Neural Computation*, vol. 1, no. 4, pp. 541–551, 1989. [Online]. Available: <https://doi.org/10.1162/neco.1989.1.4.541>
- [2] J. a. P. L. de Carvalho, J. E. Moreira, and J. N. Amaral, "Compiling for the ibm matrix engine for enterprise workloads," *IEEE Micro*, vol. 42, no. 5, pp. 34–40, 2022. [Online]. Available: <https://doi.org/10.1109/MM.2022.3176529>
- [3] N. P. Jouppi, C. Young, N. Patil, D. Patterson, G. Agrawal, R. Bajwa, S. Bates, S. Bhatia, N. Boden, A. Borchers, R. Boyle, P.-I. Cantin, C. Chao, C. Clark, J. Coriell, M. Daley, M. Dau, J. Dean, B. Gelb, T. V. Ghaemmhami, R. Gottipati, W. Gulland, R. Hagmann, C. R. Ho, D. Hogberg, J. Hu, R. Hundt, D. Hurt, J. Ibarz, A. Jaffey, A. Jaworski, A. Kaplan, H. Khaitan, D. Killebrew, A. Koch, N. Kumar, S. Lacy, J. Laudon, J. Law, D. Le, C. Leary, Z. Liu, K. Lucke, A. Lundin, G. MacKean, A. Maggiore, M. Mahony, K. Miller, R. Nagarajan, R. Narayanaswami, R. Ni, K. Nix, T. Norrie, M. Omernick, N. Penukonda, A. Phelps, J. Ross, M. Ross, A. Salek, E. Samadiani, C. Severn, G. Sizikov, M. Snellman, J. Souter, D. Steinberg, A. Swing, M. Tan, G. Thorson, B. Tian, H. Toma, E. Tuttle, V. Vasudevan, R. Walter, W. Wang, E. Wilcox, and D. H. Yoon, "In-datacenter performance analysis of a tensor processing unit," in *International Symposium on Computer Architecture*. New York, NY, USA: Association for Computing Machinery (ACM), 2017, pp. 1–12. [Online]. Available: <https://doi.org/10.1145/3079856.3080246>
- [4] J. Choquette, W. Gandhi, O. Giroux, N. Stam, and R. Krashinsky, "Nvidia a100 tensor core gpu: Performance and innovation," *IEEE Micro*, vol. 41, no. 2, pp. 29–35, 2021. [Online]. Available: <https://doi.org/10.1109/MM.2021.3061394>
- [5] J. Ansel, E. Yang, H. He, N. Gimelshein, A. Jain, M. Voznesensky, B. Bao, P. Bell, D. Berard, E. Burovski, G. Chauhan, A. Chourdia, W. Constable, A. Desmaison, Z. DeVito, E. Ellison, W. Feng, J. Gong, M. Gschwind, B. Hirsh, S. Huang, K. Kalambarakar, L. Kirsch, M. Lazos, M. Lezcano, Y. Liang, J. Liang, Y. Lu, C. K. Luk, B. Maher, Y. Pan, C. Puhersch, M. Reso, M. Saroufim, M. Y. Siraichi, H. Suk, S. Zhang, M. Suo, P. Tillet, X. Zhao, E. Wang, K. Zhou, R. Zou, X. Wang, A. Mathews, W. Wen, G. Chanan, P. Wu, and S. Chintala, "Pytorch 2: Faster machine learning through dynamic python bytecode transformation and graph compilation," in *International Conference on Architectural Support for Programming Languages and Operating Systems*. New York, NY, USA: Association for Computing Machinery (ACM), 2024, pp. 929–947. [Online]. Available: <https://doi.org/10.1145/3620665.3640366>
- [6] O. R. developers, "Onnx runtime," <https://onnxruntime.ai/>, 2021, version: 1.21.0.
- [7] K. Chellapilla, S. Puri, and P. Simard, "High performance convolutional neural networks for document processing," in *International Workshop on Frontiers in Handwriting Recognition*. La Baule (France): Suvisoft, 2006. [Online]. Available: <https://inria.hal.science/inria-00112631>
- [8] F. G. Van Zee and R. A. van de Geijn, "Blis: A framework for rapidly instantiating blas functionality," *ACM Trans. Math. Software*, vol. 41, no. 3, Jun. 2015. [Online]. Available: <https://doi.org/10.1145/2764454>
- [9] Q. Wang, X. Zhang, Y. Zhang, and Q. Yi, "Augem: automatically generate high performance dense linear algebra kernels on x86 cpus," in *International Conference on High Performance Computing, Networking, Storage and Analysis*. New York, NY, USA: Association for Computing Machinery (ACM), 2013. [Online]. Available: <https://doi.org/10.1145/2503210.2503219>
- [10] K. Goto and R. A. v. d. Geijn, "Anatomy of high-performance matrix multiplication," *ACM Trans. Math. Software*, vol. 34, no. 3, May 2008. [Online]. Available: <https://doi.org/10.1145/1356052.1356053>
- [11] M. Cho and D. Brand, "Mec: memory-efficient convolution for deep neural network," in *International Conference on Machine Learning (ICML)*. USA: JMLR.org, 2017, pp. 815–824. [Online]. Available: <https://doi.org/10.5555/3305381.3305466>
- [12] A. Ofir and G. Ben-Artzi, "SMM-Conv: Scalar matrix multiplication with zero packing for accelerated convolution," in *Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*. Los Alamitos, CA, USA: IEEE Computer Society Press, Jun. 2022, pp. 3066–3074. [Online]. Available: <https://doi.ieeecomputersociety.org/10.1109/CVPRW56347.2022.00346>
- [13] A. Anderson, A. Vasudevan, C. Keane, and D. Gregg, "High-performance low-memory lowering: Gemm-based algorithms for dnn convolution," in *International Symposium on Computer Architecture and High Performance Computing (SBAC-PAD)*. New York, NY, USA: IEEE Computer Society Press, 2020, pp. 99–106. [Online]. Available: <https://doi.org/10.1109/SBAC-PAD49847.2020.00024>
- [14] I. Korostelev, J. a. P. L. De Carvalho, J. Moreira, and J. N. Amaral, "Yaconv: Convolution with low cache footprint," *ACM Trans. Archit. Code Optim.*, vol. 20, no. 1, Feb. 2023. [Online]. Available: <https://doi.org/10.1145/3570305>
- [15] Q. Wang, S. Mei, J. Liu, and C. Gong, "Parallel convolution algorithm using implicit matrix multiplication on multi-core cpus," in *International Joint Conference*

- on *Neural Networks (IJCNN)*. New York, NY, USA: IEEE Computer Society Press, 2019, pp. 1–7. [Online]. Available: <https://doi.org/10.1109/IJCNN.2019.8852012>
- [16] M. Dukhan, “The indirect convolution algorithm,” 2019. [Online]. Available: <http://arxiv.org/abs/1907.02129>
- [17] P. San Juan, A. Castelló, M. F. Dolz, P. Alonso-Jordá, and E. S. Quintana-Ortí, “High performance and portable convolution operators for multicore processors,” in *International Symposium on Computer Architecture and High Performance Computing (SBAC-PAD)*. New York, NY, USA: IEEE Computer Society Press, 2020, pp. 91–98. [Online]. Available: <https://doi.org/10.1109/SBAC-PAD49847.2020.00023>
- [18] T. Zhao, Q. Hu, X. He, W. Xu, J. Wang, C. Leng, and J. Cheng, “Ecbc: Efficient convolution via blocked columnizing,” *IEEE Transactions on Neural Networks and Learning Systems*, vol. 34, no. 1, pp. 433–445, 2023. [Online]. Available: <https://doi.org/10.1109/TNNLS.2021.3095276>
- [19] L. Alvarenga, V. Ferrari, R. Souza, M. Pereira, and G. Araujo, “Convbench: A comprehensive benchmark for 2d convolution primitive evaluation,” 2024. [Online]. Available: <https://arxiv.org/abs/2407.10730>
- [20] C. Salvador Rohwedder, N. Henderson, J. a. P. L. De Carvalho, Y. Chen, and J. N. Amaral, “To pack or not to pack: A generalized packing analysis and transformation,” in *International Symposium on Code Generation and Optimization (CGO)*. New York, NY, USA: Association for Computing Machinery (ACM), 2023, pp. 14–27. [Online]. Available: <https://doi.org/10.1145/3579990.3580024>
- [21] M. Abadi, A. Agarwal, P. Barham, E. Brevdo, Z. Chen, C. Citro, G. S. Corrado, A. Davis, J. Dean, M. Devin, S. Ghemawat, I. Goodfellow, A. Harp, G. Irving, M. Isard, Y. Jia, R. Jozefowicz, L. Kaiser, M. Kudlur, J. Levenberg, D. Mané, R. Monga, S. Moore, D. Murray, C. Olah, M. Schuster, J. Shlens, B. Steiner, I. Sutskever, K. Talwar, P. Tucker, V. Vanhoucke, V. Vasudevan, F. Viégas, O. Vinyals, P. Warden, M. Wattenberg, M. Wicke, Y. Yu, and X. Zheng, “TensorFlow: Large-scale machine learning on heterogeneous systems,” 2015. [Online]. Available: <https://www.tensorflow.org/>
- [22] J. Möller, L. Pirch, F. Weissberg, S. Baunsgaard, T. Eisenhofer, and K. Rieck, “Adversarial inputs for linear algebra backends,” in *International Conference on Machine Learning (ICML)*, ser. Proceedings of Machine Learning Research, A. Singh, M. Fazel, D. Hsu, S. Lacoste-Julien, F. Berkenkamp, T. Maharaj, K. Wagstaff, and J. Zhu, Eds., vol. 267. PMLR, 13–19 Jul 2025, pp. 44 615–44 626. [Online]. Available: <https://proceedings.mlr.press/v267/moller25a.html>
- [23] V. Fedyunin, “Channels last memory format in pytorch,” 2020. [Online]. Available: <https://pytorch.org/tutorials/intermediate/memory%5Fformat%5Ftutorial.html>
- [24] T. maintainers and contributors, “Torchvision: Pytorch’s computer vision library,” 2016. [Online]. Available: <https://github.com/pytorch/vision>
- [25] R. Wightman, “Timm: Pytorch image models,” 2019. [Online]. Available: <https://github.com/huggingface/pytorch-image-models>
- [26] A. Howard, M. Sandler, B. Chen, W. Wang, L.-C. Chen, M. Tan, G. Chu, V. Vasudevan, Y. Zhu, R. Pang, H. Adam, and Q. Le, “Searching for MobileNetV3,” in *International Conference on Computer Vision (ICCV)*. Los Alamitos, CA, USA: IEEE Computer Society, Nov. 2019, pp. 1314–1324. [Online]. Available: <https://doi.ieeecomputersociety.org/10.1109/ICCV.2019.00140>
- [27] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *Conference on Computer Vision and Pattern Recognition (CVPR)*. New York, NY, USA: IEEE Computer Society Press, 2016, pp. 770–778. [Online]. Available: <https://doi.org/10.1109/CVPR.2016.90>
- [28] M. Tan and Q. Le, “EfficientNet: Rethinking model scaling for convolutional neural networks,” in *International Conference on Machine Learning (ICML)*, ser. Proceedings of Machine Learning Research, K. Chaudhuri and R. Salakhutdinov, Eds., vol. 97. PMLR, 09–15 Jun 2019, pp. 6105–6114. [Online]. Available: <https://proceedings.mlr.press/v97/tan19a.html>
- [29] Y. Jia, E. Shelhamer, J. Donahue, S. Karayev, J. Long, R. Girshick, S. Guadarrama, and T. Darrell, “Caffe: Convolutional architecture for fast feature embedding,” in *International Conference on Multimedia (MM)*. New York, NY, USA: Association for Computing Machinery (ACM), 2014, pp. 675–678. [Online]. Available: <https://doi.org/10.1145/2647868.2654889>
- [30] Google, “Benchmark,” 2016. [Online]. Available: <https://github.com/google/benchmark>
- [31] T. Gleixner and I. Molnar, “Linux perf,” 2009. [Online]. Available: <https://perfwiki.github.io/main/>
- [32] W. Yang, J. Fang, D. Dong, X. Su, and Z. Wang, “Libshalom: optimizing small and irregular-shaped matrix multiplications on armv8 multi-cores,” in *International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*. New York, NY, USA: Association for Computing Machinery (ACM), 2021. [Online]. Available: <https://doi.org/10.1145/3458817.3476217>
- [33] J. Li, Z. Qin, Y. Mei, J. Cui, Y. Song, C. Chen, Y. Zhang, L. Du, X. Cheng, B. Jin, Y. Zhang, J. Ye, E. Lin, and D. Lavery, “onednn graph compiler: A hybrid approach for high-performance deep learning compilation,” in *International Symposium on Code Generation and Optimization (CGO)*. New York, NY, USA: IEEE Computer Society Press, 2024, pp. 460–470. [Online]. Available: <https://doi.org/10.1109/CGO57630.2024.10444871>
- [34] A. Heinecke, G. Henry, M. Hutchinson, and H. Pabst, “Libxsmm: accelerating small matrix multiplications by runtime code generation,” in *International Conference for High Performance Computing, Networking, Storage*

and Analysis (SC). New York, NY, USA: IEEE Computer Society Press, 2016. [Online]. Available: <https://doi.org/10.5555/3014904.3015017>

- [35] D. Das, S. Avancha, D. Mudigere, K. Vaidyanathan, S. Sridharan, D. D. Kalamkar, B. Kaul, and P. Dubey, “Distributed deep learning using synchronous stochastic gradient descent,” 2016. [Online]. Available: <http://arxiv.org/abs/1602.06709>
- [36] A. Lavin and S. Gray, “Fast algorithms for convolutional neural networks,” in *Conference on Computer Vision and Pattern Recognition (CVPR)*. New York, NY, USA: IEEE Computer Society Press, 2016, pp. 4013–4021. [Online]. Available: <https://doi.org/10.1109/CVPR.2016.435>
- [37] A. Zlateski, Z. Jia, K. Li, and F. Durand, “The anatomy of efficient fft and winograd convolutions on modern cpus,” in *International Conference on Supercomputing (ICS)*. New York, NY, USA: Association for Computing Machinery (ACM), 2019, pp. 414–424. [Online]. Available: <https://doi.org/10.1145/3330345.3330382>
- [38] L. He, Y. Zhao, R. Gao, Y. Du, and L. Du, “SFC: Achieve accurate fast convolution under low-precision arithmetic,” in *International Conference on Machine Learning (ICML, ser. Proceedings of Machine Learning Research)*, R. Salakhutdinov, Z. Kolter, K. Heller, A. Weller, N. Oliver, J. Scarlett, and F. Berkenkamp, Eds., vol. 235. PMLR, 21–27 Jul 2024, pp. 18 081–18 093. [Online]. Available: <https://proceedings.mlr.press/v235/he24m.html>
- [39] J. Zhang, F. Franchetti, and T. M. Low, “High performance zero-memory overhead direct convolutions,” in *International Conference on Machine Learning (ICML)*, J. Dy and A. Krause, Eds., vol. 80. USA: PMLR, 10–15 Jul 2018, pp. 5776–5785. [Online]. Available: <https://proceedings.mlr.press/v80/zhang18d.html>
- [40] L. Zheng, C. Jia, M. Sun, Z. Wu, C. H. Yu, A. Haj-Ali, Y. Wang, J. Yang, D. Zhuo, K. Sen, J. E. Gonzalez, and I. Stoica, “Ansor: generating high-performance tensor programs for deep learning,” in *Symposium on Operating Systems Design and Implementation (OSDI)*. USA: USENIX Association, 2020. [Online]. Available: <https://www.usenix.org/conference/osdi20/presentation/zheng>
- [41] V. Ferrari, R. Sousa, M. Pereira, J. a. P. L. De Carvalho, J. N. Amaral, J. Moreira, and G. Araujo, “Advancing direct convolution using convolution slicing optimization and isa extensions,” *ACM Trans. Archit. Code Optim.*, vol. 20, no. 4, Dec. 2023. [Online]. Available: <https://doi.org/10.1145/3625004>
- [42] L. Chen, G. Papandreou, F. Schroff, and H. Adam, “Rethinking atrous convolution for semantic image segmentation,” 2017. [Online]. Available: <http://arxiv.org/abs/1706.05587>
- [43] L. Chen, Y. Zhu, G. Papandreou, F. Schroff, and H. Adam, “Encoder-decoder with atrous separable convolution for semantic image segmentation,” 2018. [Online]. Available: <http://arxiv.org/abs/1802.02611>
- [44] G. Fang, “Deeplabv3plus-pytorch,” 2020. [Online]. Available: <https://github.com/VainF/DeepLabV3Plus-Pytorch>

Algorithm 2 Extended ZConv algorithm.

Require: $IC_{gr} \leftarrow IC/GR$ $OC_{gr} \leftarrow OC/GR$
 $image[N, IH, IW, IC]$ $filter[FH, FW, IC_{gr}, OC]$
 $output[N, OH, OW, OC]$ $bias[OC]$

```

1: for  $n \leftarrow 0$  to  $N - 1$  do ▷ Parallel loop
2:   for  $ow \leftarrow 0$  to  $OW - 1$  do ▷ Parallel loop
3:     Initialize output tile (same as Algorithm 1)
4:     Allocate  $A$  with size  $OH \cdot FW \cdot IC_{gr}$ 
5:     for  $fh \leftarrow 0$  to  $FH - 1$  do
6:       for  $gr \leftarrow 0$  to  $GR - 1$  do
7:          $startH \leftarrow fh \cdot DH$ ,  $endW \leftarrow startH + OH \cdot SH$ 
8:          $startW \leftarrow ow \cdot SW$ ,  $endH \leftarrow startW + FH \cdot DW$ 
9:          $A \leftarrow image[n, startH : endH : SH, startW : endW : DW, gr \cdot IC_{GR} : (gr + 1) \cdot IC_{GR}]$ 
10:         $B \leftarrow \text{addr}(filter[fh, 0, 0, gr \cdot OC_{gr}])$ 
11:         $C \leftarrow \text{addr}(output[n, 0, ow, gr \cdot OC_{gr}])$ 
12:         $layout \leftarrow \text{rowMajor}$ 
13:         $transa \leftarrow \text{noTrans}$ ,  $transb \leftarrow \text{noTrans}$ 
14:         $m \leftarrow OH$ ,  $k \leftarrow FW \cdot IC_{gr}$ ,  $n \leftarrow OC_{gr}$ 
15:         $lda \leftarrow k$ ,  $ldb \leftarrow OC$ ,  $ldc \leftarrow OW \cdot OC$ 
16:         $\alpha \leftarrow 1$ ,  $\beta \leftarrow 1$ 
17:         $C \leftarrow \alpha AB + \beta C$  ▷ GEMM call
18:      end for
19:    end for
20:  end for
21: end for
```

- [45] M. Everingham, L. Van Gool, C. K. I. Williams, J. Winn, and A. Zisserman, “The pascal visual object classes (voc) challenge,” *International Journal of Computer Vision*, vol. 88, no. 2, pp. 303–338, Jun. 2010. [Online]. Available: <https://doi.org/10.1007/s11263-009-0275-4>

APPENDIX

Algorithm 2 presents the ZConv algorithm extended to support dilated and grouped convolution. As in Algorithm 1, padding is omitted for readability. The following changes are introduced in the extended version to support dilated and grouped convolution: (i) An extra buffer of shape $[OH, FW \times IC/GR]$ is allocated at Line 4. (ii) A new loop over the number of groups is added in Line 6. (iii) A copy operation from the image tensor to the extra buffer is added before the GEMM call. This copy handles stride, groups, and dilation, resulting in a contiguous matrix A . (iv) The GEMM parameters are adjusted for the extra buffer and to handle groups in matrices B and C .

The extra buffer A (Line 4) is necessary to support dilated and grouped convolution. Recall that in a row-major GEMM, rows must be contiguous in memory, and in the original algorithm, matrix A has a row size of $FW \times IC$. Dilation and grouping introduce irregular memory access patterns that break this contiguity. Dilation in the width dimension inserts offsets between the FW elements of a row, while grouping reduces the number of input channels accessed, such that only IC/GR elements are accessed per row instead of the full IC dimension.

Unlike in the original algorithm, matrix B points to a non-contiguous tile of the filter tensor in the grouped convolution case. When the number of groups is greater than one, each

TABLE X: Inference times on Pascal VOC 2012 (1449 images) with *DeepLabV3+*.

Backbone	PyTorch (s)	PyTorch-ZConv (s)	Speedup
<i>Xception</i>	213	189	1.13
<i>Mobilenet</i>	92	89	1.03
<i>ResNet50</i>	269	216	1.25
<i>ResNet101</i>	341	289	1.18
<i>HRNetV2_32</i>	589	456	1.29
<i>HRNetV2_48</i>	883	678	1.30

GEMM call processes only a subset of the output channels dimension (OC). Specifically, OC/GR per group, rather than the full output channel dimension. As a result, the base pointers to matrices B and C include an offset to the starting position of the current group, as shown in Lines 10 and 11, and the corresponding row offsets of these matrices are handled through the leading dimension parameters ldb and ldc .

A. *DeepLabV3+ Model Evaluation*

The first three models in 4a stand out for obtaining the highest speedups with ZConv. These bars correspond to the following models: *deeplabv3_mobilenet_v3_large*, *deeplabv3_resnet50*, and *deeplabv3_resnet101*. *DeepLabV3* is a segmentation model architecture that utilizes dilated convolution and supports distinct backbones, such as *resnet* and *mobilenet* [42]. Dilated convolutional layers typically have longer execution times due to their poor memory access patterns, as the elements within an input patch are non-contiguous in memory. In *DeepLabV3* models, dilated layers also have large padding sizes. ZConv’s approach is to gather the necessary input elements into a contiguous buffer before executing a GEMM call. While *Im2col* has a similar approach, ZConv minimizes the number of non-contiguous accesses because it only considers the width dilation in the copy to the new buffer. The height dilation only affects the loop that iterates over the filter height (FH). Combined with its efficient padding handling, as discussed in Section IV-B3, ZConv significantly improves performance for these layers.

Table X extends the evaluation to the more recent *DeepLabV3+* models [43]. While Timm and TorchVision do not provide these models, their PyTorch implementations are available on GitHub [44]. Unlike the previous end-to-end model evaluations, these models are evaluated following the usage of the repository that implements them. Thus, each model is run on a validation set of the Pascal VOC 2012 dataset, which contains 1449 images [45]. Table X presents details on the evaluated *DeepLabV3+* model backbones, the total inference time for multithreaded execution across all validation images, and the corresponding speedup obtained by PyTorch-ZConv. For a simpler integration, the weight layouts remained unchanged after converting the model to the channel-last format, requiring ZConv to convert the weight tensor to (FH, FW, IC, OC) before execution. Despite this additional transformation overhead, ZConv improves execution time across all models, further demonstrating its effectiveness for this architecture.

The supplementary material provided with this submission includes: (1) The figures and tables from the paper, and the raw data used to generate them. (2) Source code for the ZConv implementations (standalone and PyTorch integration). (3) Benchmarking and evaluation scripts to reproduce measurements. (4) A Dockerfile that details how to build all components required for the evaluation environment. Due to size restrictions, a docker image is not included, but will be made available when this material is packaged into an archival format.

Full reproduction instructions included in the artifact README. The artifact is anonymized and contains no public repository URLs or author-identifying information; upon acceptance we will publish the full artifact (code, docker image, and evaluation scripts) with a public archive and DOI.